

Reineke-RAG

Technische Dokumentation

Lokales, multi-tenant Retrieval-Augmented-Generation-System
(rag-qdrant-local)

100 % offline · on-premise

Reineke-Technik

Stand 28. April 2026

Code-Stand: 17 Module · 3.036 Python-Zeilen · 9 HTTP-Endpunkte

1 · Überblick

Reineke-RAG ist ein vollständig **offline** betriebenes Retrieval-Augmented-Generation-System für die Beantwortung natürlichsprachiger Fragen über interne Dokumente (PDF, Word, Excel — auch Legacy-Formate `.doc` und `.xls`). Die aktuelle Codebasis liegt unter `rag-qdrant-local/` und umfasst rund **3.000 Zeilen Python** in einem FastAPI-Backend.

Eigenschaft	Wert
Einsatzart	Server-seitig, vollständig on-premise
Internet-Zugriff	nicht erforderlich
Quell-Dokumente	Dateisystem (kein Upload-Workflow)
Unterstützte Formate	PDF, DOCX, DOC, XLSX, XLS
Mandantenfähig	ja (Tenant + Project)
Halluzinationsschutz	ja, durch Score-Schwelle und Quellenzwang
LLM-Laufzeit	Ollama (Apple-Silicon-tauglich)
Vektor-DB	Qdrant
Metadaten-Store	SQLite (WAL-Mode)
API-Stil	FastAPI, OpenAPI 3, OpenAI-kompatibel

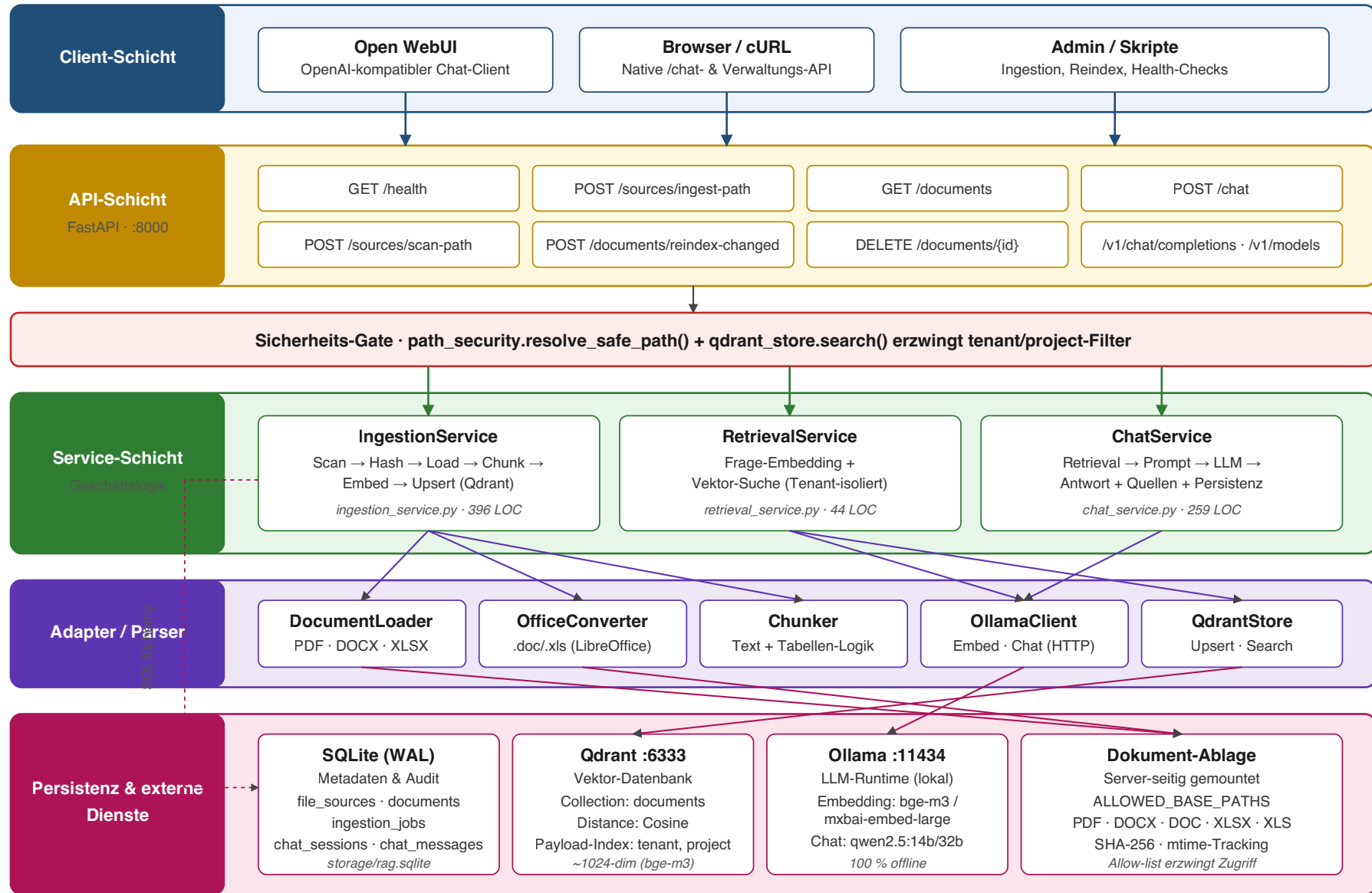
Das System ist so konzipiert, dass es ausschließlich auf vom Administrator freigegebenen Verzeichnis bäumen arbeitet. Datei-Inhalte verlassen die Maschine nicht.

2 · Architektur

Das nachfolgende Schema (eine Seite, separat enthalten) zeigt die fünf Schichten — Client, API, Sicherheits-Gate, Service, Adapter/Parser sowie Persistenz/externe Dienste — und die Datenflüsse zwischen ihnen.

Reineke-RAG · System-Architektur (Offline)

Lokale, multi-tenant Retrieval-Augmented-Generation Pipeline · Stand 2026-04-28



© Reineke-Technik · alle Daten on-premise

Schichten in Kurzform

1. **Client-Schicht** — Open WebUI, Browser/cURL, Admin-Skripte
 2. **API-Schicht** — FastAPI auf Port 8000, native Endpunkte plus OpenAI-Kompatibilität
 3. **Sicherheits-Gate** — Pfad-Allow-list und erzwungene Mandanten-Filter
 4. **Service-Schicht** — `IngestionService`, `RetrievalService`, `ChatService`
 5. **Adapter & Parser** — Dokumenten-Loader, Office-Konverter, Chunker, Ollama-Client, Qdrant-Store
 6. **Persistenz & externe Dienste** — SQLite, Qdrant (`:6333`), Ollama (`:11434`), Dateisystem
-

3 · Technologie-Komponenten

3.1 Laufzeit & Frameworks

Komponente	Version	Zweck
Python	3.11 (Container) / 3.12 (Tests)	Programmiersprache
FastAPI	0.115.0	HTTP-API, OpenAPI-Spezifikation
Uvicorn	0.30.6	ASGI-Server
Pydantic	2.9.2	Datenvalidierung, Schemata
pydantic-settings	2.5.2	<code>.env</code> -Konfiguration
python-dotenv	1.0.1	Environment-Loader
httpx	0.27.2	Async HTTP-Client (Ollama)
SQLAlchemy	2.0.36	ORM für SQLite
aiosqlite	0.20.0	Async-Treiber
qdrant-client	1.12.1	Qdrant SDK
python-multipart	0.0.12	Multipart-Form-Daten

3.2 Dokumenten-Parsing

Format	Parser	Strategie
<code>.pdf</code>	pypdf 5.0.1	seitenweise Text-Extraktion, Erkennung Image-only PDFs
<code>.docx</code>	python-docx 1.1.2	Paragraphen + Tabellen → Markdown
<code>.xlsx</code>	openpyxl 3.1.5	header-erkennend, zeilenbasiertes Chunking
<code>.doc</code>	LibreOffice (headless) → <code>.docx</code>	über <code>office_converter.py</code>
<code>.xls</code>	LibreOffice (headless) → <code>.xlsx</code>	über <code>office_converter.py</code>

3.3 KI-Komponenten (lokal)

Aufgabe	Modell-Default	Empfohlen	Dimension
Embedding	<code>mxbai-embed-large</code>	<code>bge-m3</code> (mehrsprachig)	1024
Chat-Generierung	<code>qwen2.5:14b</code>	<code>qwen2.5:32b-instruct-q4_K_M</code>	—

Beide Modelle laufen über **Ollama** (<http://localhost:11434>) — keine Cloud-Anbindung.

3.4 Datenhaltung

Speicher	Pfad / Endpunkt	Zweck
SQLite	<code>storage/rag.sqlite</code> (WAL)	Metadaten, Audit, Chat-Verlauf
Qdrant	<code>http://localhost:6333</code>	Vektor-Index (Cosine)
Dateisystem	<code>ALLOWED_BASE_PATHS</code>	Original-Dokumente (read-only)

3.5 Container & Deployment

- **Dockerfile** vorhanden (`backend/Dockerfile`) — Python 3.11-slim + LibreOffice + curl.
- Externe Dienste (Qdrant, Ollama) werden als Side-Cars / separate Container betrieben.
- Volumes: `storage/rag.sqlite` , `storage/converted/` , `storage/temp/` .

4 · Funktionsverzeichnis

Das Backend ist in **17 Python-Module** unter `backend/app/` aufgeteilt. Die folgenden Tabellen listen sämtliche öffentlichen Funktionen, Klassen und Methoden auf (strukturiert nach Modul).

4.1 `config.py` — Konfiguration

Symbol	Typ	Zweck
<code>Settings</code>	Klasse	Pydantic- <code>BaseSettings</code> mit allen <code>.env</code> -Werten
<code>Settings.allowed_base_paths</code>	Property	Parst <code>ALLOWED_BASE_PATHS</code> als <code>List[Path]</code>
<code>Settings.sqlite_path</code>	Property	Aufgelöster Pfad zur SQLite-Datei
<code>Settings.converted_dir</code>	Property	Verzeichnis für Office-Konvertierungsergebnisse
<code>Settings.temp_dir</code>	Property	Verzeichnis für temporäre Dateien
<code>_project_root() -> Path</code>	Funktion	Ermittelt Projekt-Root unabhängig vom CWD
<code>get_settings() -> Settings</code>	Funktion	Cached Singleton (<code>@lru_cache</code>)

4.2 `database.py` — SQLite Engine & Sessions

Symbol	Signatur	Zweck
<code>_build_engine()</code>	<code>() -> Engine</code>	Erstellt SQLAlchemy-Engine mit WAL + 30-s-Busy-Timeout
<code>init_db()</code>	<code>() -> None</code>	Erstellt alle Tabellen aus <code>models.Base.metadata</code>
<code>session_scope()</code>	<code>() -> Iterator[Session]</code>	Context-Manager mit Commit/Rollback
<code>get_db()</code>	<code>() -> Iterator[Session]</code>	FastAPI-Dependency

4.3 `models.py` — ORM-Klassen

Klasse	Tabelle	Wichtige Felder
<code>FileSource</code>	<code>file_sources</code>	<code>id</code> · <code>tenant</code> · <code>project</code> · <code>base_path</code> · <code>recursive</code> · <code>created_at</code> · <code>last_scan_at</code> · <code>last_ingest_at</code>
<code>Document</code>	<code>documents</code>	<code>id</code> · <code>tenant</code> · <code>project</code> · <code>source_path</code> · <code>file_name</code> · <code>file_extension</code> · <code>file_size</code> · <code>checksum</code> · <code>modified_at</code> · <code>status</code> · <code>chunks_count</code> · <code>error_message</code> · <code>created_at</code> · <code>updated_at</code>
<code>IngestionJob</code>	<code>ingestion_jobs</code>	<code>id</code> · <code>tenant</code> · <code>project</code> · <code>source_path</code> · <code>status</code> · <code>files_found</code> · <code>files_indexed</code> · <code>files_skipped</code> · <code>files_failed</code> · <code>chunks_created</code> · <code>error_message</code> · <code>created_at</code> · <code>completed_at</code>
<code>ChatSession</code>	<code>chat_sessions</code>	<code>id</code> · <code>tenant</code> · <code>project</code> · <code>created_at</code> · <code>messages</code> (1:n)
<code>ChatMessage</code>	<code>chat_messages</code>	<code>id</code> · <code>session_id</code> · <code>role</code> · <code>content</code> · <code>sources_json</code> · <code>created_at</code>

`Document.status` ∈ { `pending` , `indexed` , `failed` , `deleted` , `requires_ocr` }.

4.4 `schemas.py` — Pydantic-DTOs

Schema	Zweck
<code>TenantProject</code>	Mixin für <code>tenant</code> + <code>project</code>
<code>FileEntry</code>	Datei aus Scan-Resultat
<code>HealthCheckItem</code> , <code>HealthResponse</code>	<code>/health</code> -Antwort
<code>ScanPathRequest</code> , <code>ScanPathResponse</code>	<code>/sources/scan-path</code>
<code>IngestPathRequest</code> , <code>IngestPathResponse</code> , <code>IngestError</code>	<code>/sources/ingest-path</code>
<code>ReindexChangedRequest</code>	<code>/documents/reindex-changed</code>
<code>DocumentOut</code> , <code>DocumentListResponse</code>	<code>GET /documents</code>
<code>DeleteDocumentResponse</code>	<code>DELETE /documents/{id}</code>
<code>ChatRequest</code> , <code>ChatSource</code> , <code>ChatResponse</code>	<code>/chat</code>
<code>OpenAIMessage</code> , <code>OpenAIChatCompletionRequest</code> , <code>OpenAIChatCompletionResponse</code> , <code>OpenAIModelEntry</code> , <code>OpenAIModelList</code>	OpenAI-Kompatibilität

4.5 `main.py` — FastAPI-App & Endpunkte

Endpunkt	Methode	Funktion	Beschreibung
<code>/health</code>	GET	<code>health()</code>	Pingt Ollama, Qdrant, Modelle
<code>/sources/scan-path</code>	POST	<code>sources_scan_path()</code>	Scannt Verzeichnis ohne zu indexieren
<code>/sources/ingest-path</code>	POST	<code>sources_ingest_path()</code>	Vollständiger Ingest mit Embedding + Upsert
<code>/documents</code>	GET	<code>list_documents()</code>	Listet indizierte Dokumente
<code>/documents/reindex-changed</code>	POST	<code>reindex_changed()</code>	Reindex nur geänderter Dateien
<code>/documents/{document_id}</code>	DELETE	<code>delete_document()</code>	Löscht Dokument + Vektoren
<code>/chat</code>	POST	<code>chat_endpoint()</code>	RAG-Frage/Antwort mit Quellen
<code>/v1/models</code>	GET	<code>openai_models()</code>	OpenAI-kompatible Modell-Liste
<code>/v1/chat/completions</code>	POST	<code>openai_chat_completions()</code>	OpenAI-kompatibler Chat-Endpoint

Hilfsfunktionen: `get_ingestion()`, `get_chat()` (Dependency-Injektoren).

4.6 `path_security.py` — Pfad-Sicherheit

Symbol	Signatur	Zweck
<code>PathSecurityError</code>	Exception	Eingabe nicht erlaubt
<code>_is_within(child, parent)</code>	<code>(Path, Path) -> bool</code>	Prüft Eltern-Beziehung
<code>_is_under_system_deny(path, allowed_bases)</code>	<code>(Path, List[Path]) -> bool</code>	Schutz vor <code>/etc</code> , <code>/root</code> etc.
<code>get_allowed_base_paths()</code>	<code>() -> List[Path]</code>	Liest und validiert <code>ALLOWED_BASE_PATHS</code>
<code>resolve_safe_path(user_input)</code>	<code>(str) -> Path</code>	Hauptfunktion: kanonisiert + prüft
<code>assert_existing_dir(path)</code>	<code>(Path) -> Path</code>	Prüft Existenz und Verzeichnis-Typ

System-Deny-Liste (Default): `/etc`, `/root`, `/home`, `/var`, `/proc`, `/sys`, `/dev`, `/boot`, `/usr`, `/bin`, `/sbin`, `/lib`, `/opt`.

4.7 `source_scanner.py` — Datei-Scanner

Symbol	Signatur	Zweck
<code>SUPPORTED_EXTENSIONS</code>	Konstante	<code>{.pdf, .docx, .doc, .xlsx, .xls}</code>
<code>_IGNORED_DIR_NAMES</code>	Konstante	<code>.git</code> , <code>__pycache__</code> , <code>node_modules</code> , ...
<code>ScanResult</code>	Dataclass	Felder: <code>supported</code> , <code>unsupported</code> , <code>file_types</code>
<code>_iter_files(root, recursive)</code>	Iterator	Datei-Iteration mit Ignore-Liste
<code>_is_in_ignored_dir(p)</code>	<code>(Path) -> bool</code>	True, falls Ordner ignoriert
<code>_classify(path)</code>	<code>(Path) -> tuple[bool, str]</code>	(unterstützt?, Endung)
<code>scan_directory(root, *, recursive=True)</code>	<code>(Path) -> ScanResult</code>	Hauptfunktion

4.8 `document_loader.py` — Dokumenten-Parsing

Symbol	Signatur	Zweck
<code>DocumentLoadError</code>	Exception	Allgemeiner Lade-Fehler
<code>RequiresOCRError</code>	Exception	PDF ohne Text-Layer (Bild-PDF)
<code>LoadedSegment</code>	Dataclass	Ein Textblock + Metadaten
<code>LoadedDocument</code>	Dataclass	Sammlung aller Segmente eines Dokuments
<code>_load_pdf(path)</code>	<code>(Path) -> List[LoadedSegment]</code>	Seitenweises Extrahieren
<code>_table_to_markdown(table)</code>	<code>(table) -> str</code>	Word-Tabelle als Markdown
<code>_load_docx(path)</code>	<code>(Path) -> List[LoadedSegment]</code>	Paragraphen + Tabellen
<code>_load_xlsx(path)</code>	<code>(Path) -> List[LoadedSegment]</code>	Sheet-Erkennung, Header-Heuristik
<code>load_document(path)</code>	<code>(Path) -> LoadedDocument</code>	Dispatcher für alle Formate

4.9 `office_converter.py` — Legacy-Office-Konvertierung

Symbol	Signatur	Zweck
<code>OfficeConversionError</code>	Exception	LibreOffice-Fehler / fehlt
<code>libreoffice_available()</code>	<code>() -> bool</code>	Prüft <code>soffice</code> im PATH
<code>_run_soffice(src, target_format, outdir)</code>	<code>(Path, str, Path) -> Path</code>	Subprocess-Wrapper, 180 s Timeout
<code>convert_doc_to_docx(src, outdir=None)</code>	<code>(Path) -> Path</code>	<code>.doc</code> → <code>.docx</code>
<code>convert_xls_to_xlsx(src, outdir=None)</code>	<code>(Path) -> Path</code>	<code>.xls</code> → <code>.xlsx</code>

4.10 `chunker.py` — Chunking-Strategien

Symbol	Signatur	Zweck
<code>Chunk</code>	Dataclass	Embed-fähiger Textblock + Metadaten
<code>_split_paragraphs(text)</code>	<code>(str) -> List[str]</code>	Trennung an Leerzeilen
<code>_split_long_paragraph(p, size)</code>	<code>(str, int) -> List[str]</code>	Satz-bewusster Hard-Wrap
<code>_chunk_text(text, size, overlap)</code>	<code>(str, int, int) -> List[str]</code>	Sliding Window
<code>_chunk_text_segment(seg, start_index)</code>	<code>(LoadedSegment, int) -> List[Chunk]</code>	Wrapper für Text-Segmente
<code>_xlsx_rows_per_chunk(num_columns)</code>	<code>(int) -> int</code>	Adaptive Block-Größe
<code>_chunk_spreadsheet_segment(seg, start_index)</code>	<code>(LoadedSegment, int) -> List[Chunk]</code>	Tabellen-Chunks (Header wiederholt)
<code>chunk_document(doc)</code>	<code>(LoadedDocument) -> List[Chunk]</code>	Dispatcher

Default-Werte: `CHUNK_SIZE=1000`, `CHUNK_OVERLAP=150`, `XLSX_ROWS_PER_CHUNK=40`, `XLSX_MAX_CHARS_PER_CHUNK=6000`.

4.11 `ollama_client.py` — LLM-Client

Methode	Signatur	Zweck
<code>OllamaError</code>	Exception	API- oder Netzwerk-Fehler
<code>OllamaClient.__init__(base_url=None, timeout=120.0)</code>	Konstruktor	DI über Settings
<code>_request(method, path, *, json=None)</code>	async	Low-level HTTP
<code>list_models()</code>	<code>() -> List[str]</code>	GET /api/tags
<code>has_model(name)</code>	<code>(str) -> bool</code>	Prefix-Match
<code>embed(text, *, model=None)</code>	<code>(str) -> List[float]</code>	POST /api/embeddings
<code>embed_many(texts, *, model=None)</code>	<code>(List[str]) -> List[List[float]]</code>	Sequenzielle Calls
<code>chat(messages, *, model=None, temperature=None, max_tokens=None)</code>	<code>(...) -> str</code>	POST /api/chat
<code>ping()</code>	<code>() -> bool</code>	Health-Probe

4.12 `qdrant_store.py` — Vektor-Store

Methode	Signatur	Zweck
<code>QdrantStoreError</code>	Exception	Inkonsistenter Aufruf / API-Fehler
<code>SearchHit</code>	Dataclass	<code>score</code> , <code>payload</code> , <code>point_id</code>
<code>QdrantStore.__init__(url=None, api_key=None, collection=None)</code>	Konstruktor	DI über Settings
<code>ping()</code>	<code>() -> bool</code>	Erreichbarkeit
<code>ensure_collection(vector_size)</code>	<code>(int) -> None</code>	Erstellt oder prüft Collection (Cosine)
<code>_ensure_payload_indexes()</code>	<code>() -> None</code>	Idempotente Index-Erstellung (tenant, project, ...)
<code>upsert_chunks(*, document_id, vectors, payloads)</code>	<code>(...) -> int</code>	Schreibt Punkte (Wait=True)
<code>delete_document(document_id)</code>	<code>(str) -> int</code>	Löscht alle Punkte eines Dokuments
<code>search(*, tenant, project, query_vector, top_k, score_threshold=None)</code>	<code>(...) -> List[SearchHit]</code>	Pflicht-Filter: tenant + project, sonst Exception

4.13 `ingestion_service.py` — Ingest-Pipeline

Methode	Signatur	Zweck
<code>IngestionService.__init__(ollama=None, store=None)</code>	Konstruktor	DI
<code>_ensure_collection_for_model()</code>	async	Probt Embedding-Dimension, erstellt Collection
<code>_upsert_file_source(db, *, tenant, project, base_path, recursive)</code>	sync	Legt/aktualisiert <code>FileSource</code> -Datensatz
<code>_find_or_create_document(db, *, tenant, project, path, checksum)</code>	sync	(Document, is_new)
<code>ingest_path(db, *, tenant, project, path, recursive=True, reindex_changed_only=True)</code>	async	Komplette Pipeline
<code>reindex_changed(db, *, tenant, project, path, recursive=True, mark_missing_as_deleted=False)</code>	async	Differenz-Reindex
<code>delete_document(db, document_id)</code>	async	Qdrant + SQLite löschen

4.14 `retrieval_service.py` — Suche

Methode	Signatur	Zweck
<code>RetrievalService.__init__(ollama=None, store=None)</code>	Konstruktor	DI
<code>retrieve(*, tenant, project, question, top_k=None, min_score=None)</code>	async, () -> <code>List[SearchHit]</code>	Embedding + Qdrant-Suche mit Mandanten-Filter

4.15 `chat_service.py` — Chat-Orchestrierung

Methode	Signatur	Zweck
<code>SYSTEM_PROMPT</code>	Konstante	Deutsche RAG-Anweisung
<code>NO_CONTEXT_ANSWER</code>	Konstante	Fallback bei 0 Treffern
<code>ChatService.__init__(retrieval=None, ollama=None, session_factory=None)</code>	Konstruktor	DI
<code>_resolve_session_id(*, tenant, project, session_id=None)</code>	sync	DB-Phase A: Session laden/anlegen
<code>_persist_messages(*, session_id, question, answer, sources)</code>	sync	DB-Phase C: Messages persistieren
<code>_build_context(hits)</code> static	<code>(...) -> str</code>	Quellenblock aufbauen
<code>_hits_to_sources(hits)</code> static	<code>(...) -> List[ChatSource]</code>	DTO-Konversion
<code>chat(*, tenant, project, question, session_id=None, top_k=None)</code>	async, <code>() -> ChatResponse</code>	Hauptmethode

4.16 `openai_compat.py` — OpenAI-Adapter

Methode	Signatur	Zweck
<code>OpenAIRequestError</code>	Exception	Anfrage nicht mappbar
<code>ResolvedRequest</code>	Dataclass	tenant, project, question, session_id, model
<code>_last_user_message(messages)</code>	sync	Extrahiert letzte User-Nachricht
<code>_parse_model_id(model)</code>	<code>(str) -> Optional[tuple]</code>	<code>rag:tenant:project</code> parsen
<code>_coerce_str(value)</code>	sync	Type-Coercion
<code>resolve_openai_request(req)</code>	<code>(...) -> ResolvedRequest</code>	Mapping inkl. <code>extra_body</code>
<code>build_openai_response(*, model, answer, sources, session_id)</code>	<code>(...) -> OpenAIChatCompletionResponse</code>	Antwort-Aufbau

4.17 `utils.py` — Hilfsfunktionen

Funktion	Signatur	Zweck
<code>configure_logging()</code>	<code>() -> None</code>	Initialisiert Logging laut <code>LOG_LEVEL</code>
<code>get_logger(name)</code>	<code>(str) -> Logger</code>	Wrapper
<code>new_id()</code>	<code>() -> str</code>	UUID4
<code>utcnow_iso()</code>	<code>() -> str</code>	ISO-8601-Zeitstempel
<code>deterministic_uuid(*parts)</code>	<code>(...) -> str</code>	UUID5 (für Chunk-Point-IDs)
<code>sha256_file(path, *, chunk_size=1MB)</code>	<code>(Path) -> str</code>	Stream-Hash
<code>file_modified_iso(path)</code>	<code>(Path) -> str</code>	mtime als ISO
<code>chunked(items, n)</code>	<code>(...) -> Iterator</code>	Batch-Iterator

5 · HTTP-API im Detail

5.1 Native Endpunkte

GET /health

Liefert Status-Items (`backend`, `ollama`, `qdrant`, `embedding_model`, `chat_model`). Antwort enthält pro Item `{name, ok, detail}`.

POST /sources/scan-path

Body: `{tenant, project, path, recursive?}` Antwort enthält `supported`, `unsupported`, `file_types`. Indexiert **nichts**.

POST /sources/ingest-path

Body: `{tenant, project, path, recursive?, reindex_changed_only?}` Komplette Pipeline. Antwort: `{job_id, indexed_files, failed_files, chunks_created, errors[]}`.

GET /documents?tenant=...&project=...&include_deleted=false

Listet alle Dokumente eines Mandanten/Projekts mit Status und Chunk-Anzahl.

POST /documents/reindex-changed

Wie `ingest-path`, jedoch ohne neue Dateien — nur geänderte werden re-embedded. Optional `mark_missing_as_deleted=true` markiert verschwundene Dokumente.

DELETE /documents/{document_id}

Löscht alle zugehörigen Qdrant-Punkte und den SQLite-Eintrag.

POST /chat

Body: `{tenant, project, question, session_id?, top_k?}` Antwort: `{answer, sources[], session_id}`. * Wenn keine Quellen über Schwelle → Antwort = `NO_CONTEXT_ANSWER`, LLM wird **nicht** aufgerufen.

5.2 OpenAI-kompatible Endpunkte

GET /v1/models

Listet virtuelle Modelle in der Form `rag:<tenant>:<project>` (oder `rag:default`).

POST /v1/chat/completions

Akzeptiert OpenAI-Standard. Tenant/Projekt werden bestimmt aus 1. Inline-Feldern, 2. `extra_body`, 3. Modell-ID `rag:tenant:project`. Streaming wird abgelehnt (`stream=true` ⇒ Fehler).

5.3 Fehler-Klassen

HTTP	Auslöser
400	<code>PathSecurityError</code> , <code>OpenAIRequestError</code> , <code>ValueError</code>
404	Dokument nicht gefunden
500	<code>QdrantStoreError</code> , allgemeiner Fehler
502	<code>OllamaError</code> (LLM nicht erreichbar)

6 · Datenflüsse

6.1 Ingest-Pipeline

1. `POST /sources/ingest-path` → Pfad-Sicherheits-Gate.
2. `IngestionService.ingest_path()` öffnet Qdrant-Collection (Dimension passend zum Embedding-Modell).
3. `scan_directory()` liefert alle unterstützten Dateien.
4. Pro Datei: * `sha256_file()` berechnet Prüfsumme. * Bei unverändertem Hash und Status `indexed` → Skip. * `load_document()` extrahiert Segmente. * `chunk_document()` erzeugt Chunks (Text- oder Tabellen-Strategie). * `OllamaClient.embed_many()` liefert Vektoren. * `QdrantStore.upsert_chunks()` schreibt Punkte mit Tenant/Projekt-Payload. * SQLite-Status auf `indexed` (oder `failed` / `requires_ocr`).
5. Antwort enthält Job-Statistik und Fehlerliste.

6.2 Chat-Pipeline

1. `POST /chat` (oder `/v1/chat/completions`).
2. `_resolve_session_id()` lädt oder legt eine `ChatSession` an.
3. `RetrievalService.retrieve()` erzeugt Frage-Embedding und ruft `QdrantStore.search()` mit erzwungenem Tenant/Projekt-Filter und Score-Schwelle auf.
4. Bei **0 Treffern** → `NO_CONTEXT_ANSWER`, Persistierung, Rückgabe.
5. Sonst: `_build_context()` erzeugt nummerierten Quellenblock, `OllamaClient.chat()` generiert Antwort.
6. `_persist_messages()` speichert User- und Assistant-Message inkl. `sources_json`.
7. Antwort: `{answer, sources, session_id}`.

6.3 Externe Service-Abhängigkeiten

Quelle	Ziel	Protokoll	Zweck
Backend	Ollama (<code>:11434</code>)	HTTP	Embeddings, Chat
Backend	Qdrant (<code>:6333</code>)	HTTP	Upsert, Suche
Backend	Dateisystem	POSIX	Originaldokumente lesen
Backend	LibreOffice (Subprozess)	CLI	Legacy-Konvertierung

7 · Datenmodell

7.1 SQLite-Schema (5 Tabellen)

Tabelle	Schlüssel	Zweck
<code>file_sources</code>	id	Konfigurierte Wurzelpfade pro Mandant
<code>documents</code>	id (UNIQUE: tenant, project, source_path)	Indizierte Dateien
<code>ingestion_jobs</code>	id	Audit-Trail aller Ingest-Läufe
<code>chat_sessions</code>	id	Konversationscontainer
<code>chat_messages</code>	id (FK: session_id)	Einzelne Nachrichten + Quellen

WAL-Mode, `foreign_keys=ON`, `synchronous=NORMAL`, `busy_timeout=30 s`.

7.2 Qdrant-Punktformat

```
{
  id      : UUID5(document_id, chunk_index),
  vector  : [float × dim],
  payload : {
    tenant, project, document_id, file_name,
    file_extension, source_path, checksum, modified_at,
    document_type, chunk_index, page?, sheet?,
    row_start?, row_end?, text
  }
}
```

Indizierte Payload-Felder: `tenant`, `project`, `document_id`, `file_extension`.

8 · Konfiguration (`.env`)

Variable	Default	Beschreibung
<code>OLLAMA_BASE_URL</code>	<code>http://localhost:11434</code>	LLM-Endpunkt
<code>QDRANT_URL</code>	<code>http://localhost:6333</code>	Vektor-DB
<code>QDRANT_API_KEY</code>	—	Optional
<code>QDRANT_COLLECTION</code>	<code>documents</code>	Collection-Name
<code>EMBEDDING_MODEL</code>	<code>mxbai-embed-large</code>	Empfohlen: <code>bge-m3</code>
<code>CHAT_MODEL</code>	<code>qwen2.5:14b</code>	Größere Modelle möglich
<code>ALLOWED_BASE_PATHS</code>	—	Pflicht — kommaseparierte absolute Pfade
<code>SQLITE_DB_PATH</code>	<code>./storage/rag.sqlite</code>	Metadaten-DB
<code>CHUNK_SIZE</code>	<code>1000</code>	Zeichen pro Text-Chunk
<code>CHUNK_OVERLAP</code>	<code>150</code>	Überlappung
<code>XLSX_ROWS_PER_CHUNK</code>	<code>40</code>	Zeilen pro Tabellen-Chunk
<code>XLSX_MAX_CHARS_PER_CHUNK</code>	<code>6000</code>	Hardlimit
<code>RETRIEVAL_TOP_K</code>	<code>6</code>	Anzahl Treffer
<code>MIN_RETRIEVAL_SCORE</code>	<code>0.35</code>	Schwelle für „relevant“
<code>CHAT_TEMPERATURE</code>	<code>0.1</code>	Sampling
<code>CHAT_MAX_TOKENS</code>	<code>1024</code>	Max. Antwortlänge
<code>SOFFICE_BIN</code>	<code>soffice</code>	LibreOffice-Binary
<code>HOST</code>	<code>0.0.0.0</code>	FastAPI
<code>PORT</code>	<code>8000</code>	FastAPI
<code>LOG_LEVEL</code>	<code>INFO</code>	DEBUG/INFO/WARN/ERROR

9 · Sicherheits-Eigenschaften

1. **Allow-list-basierter Dateizugriff.** Ohne `ALLOWED_BASE_PATHS` lehnt das System jede Scan- oder Ingest-Anfrage ab.
 2. **Pfad-Traversal-Schutz.** `resolve_safe_path()` kanonisiert mit `Path.resolve()`, prüft Eltern-Beziehung, lehnt Systemverzeichnisse (`/etc`, `/root`, ...) ab, sofern diese nicht explizit freigegeben sind.
 3. **Mandanten-Isolation als Pflicht-Filter.** `QdrantStore.search()` wirft `QdrantStoreError`, wenn `tenant` oder `project` leer sind — eine ungefilterte Vektor-Suche ist nicht möglich.
 4. **Anti-Halluzination.** Ohne Treffer über `MIN_RETRIEVAL_SCORE` gibt das System die Konstante `NO_CONTEXT_ANSWER` zurück; das LLM wird in diesem Fall **nicht** angefragt.
 5. **Quellenzwang.** Jede generierte Antwort wird mit `sources` (Datei, Seite/Sheet/Zeilenbereich, Score) ausgeliefert und persistiert.
 6. **Offline-Betrieb.** Sämtliche Modelle laufen lokal über Ollama; das Backend macht keine ausgehenden Verbindungen außer zu Qdrant und Ollama.
 7. **Read-only auf Originale.** Ingest-Pipeline schreibt nicht in die Quellverzeichnisse. Konvertierungsergebnisse landen in `storage/converted/`.
-

10 · Tests

Test-Verzeichnis: `backend/tests/` (8 Module). Ausführung:

```
cd backend
pytest -v
```

Testdatei	Fokus
<code>test_path_security.py</code>	Allow-list, Traversal, System-Pfad-Ablehnung
<code>test_source_scanner.py</code>	Klassifikation, Recursive-Flag
<code>test_extractors.py</code>	PDF/DOCX/XLSX-Parsing
<code>test_chunker_xlsx.py</code>	XLSX-Chunking inklusive Header-Wiederholung
<code>test_qdrant_filter.py</code>	Pflicht-Filter Tenant + Projekt
<code>test_openai_compat.py</code>	OpenAI-Request-Auflösung
<code>test_chat_no_hits.py</code>	Fallback bei fehlenden Treffern (async)
<code>conftest.py</code>	Fixtures (Sandbox, Env)

End-to-End-Bericht (`TEST_REPORT.md` , 28. April 2026): * Korpus 61 Dateien, 767 Chunks, 0 Fehler nach Wechsel auf `bge-m3` . * Alle 4 Test-Fragen liefern entweder konsistenten Fallback oder präzise Antwort mit korrekter Quellenangabe.

11 · Deployment

11.1 Voraussetzungen

- Ollama ≥ 0.3 lokal laufend (Modelle vorgepullt: `bge-m3`, `qwen2.5:14b` o. ä.).
- Qdrant ≥ 1.17 erreichbar.
- LibreOffice (für `.doc` / `.xls`) — optional, falls Legacy-Formate vorkommen.

11.2 Schnellstart

```
git clone <repo>
cd rag-qdrant-local
cp .env.example .env
# In .env: ALLOWED_BASE_PATHS=/srv/dokumente,/mnt/policies ← anpassen

cd backend
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

uvicorn app.main:app --host 0.0.0.0 --port 8000
```

11.3 Beispiel-Workflow

```
# 1. Health-Check
curl http://localhost:8000/health

# 2. Scan
curl -X POST http://localhost:8000/sources/scan-path \
  -H "Content-Type: application/json" \
  -d '{"tenant":"reineke","project":"watch","path":"/srv/dokumente/watch"}'

# 3. Ingest
curl -X POST http://localhost:8000/sources/ingest-path \
  -H "Content-Type: application/json" \
  -d '{"tenant":"reineke","project":"watch","path":"/srv/dokumente/watch"}'

# 4. Frage stellen
curl -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"tenant":"reineke","project":"watch","question":"Welche Passwortregeln gelten?"}'
```

11.4 Open-WebUI-Integration

- In Open WebUI als „OpenAI-API“ eintragen, Base-URL = `http://<backend>:8000/v1`.
- Modell wählen: `rag:reineke:watch` (Format `rag:<tenant>:<project>`).
- Chat verwendet automatisch den RAG-Pfad.

11.5 Wartung

Aktion	Befehl
Reindex geänderter Dateien	<code>POST /documents/reindex-changed</code>
Verschwundene Dateien als „deleted“	<code>...?mark_missing_as_deleted=true</code>
Einzelnes Dokument entfernen	<code>DELETE /documents/{id}</code>
SQLite-Backup	<code>sqlite3 storage/rag.sqlite ".backup backup.sqlite"</code>
Qdrant-Snapshot	<code>curl -X POST http://localhost:6333/collections/documents/snapshots</code>

12 · Status & nächste Schritte

- **Reife.** End-to-End funktionsfähig, Test-Bericht 2026-04-28 grün.
- **Empfohlene Nachjustierung.**
- Default-Embedder in `.env.example` von `mxbai-embed-large` auf `bge-m3` umstellen (besser für deutsche Texte und große XLSX-Chunks).
- Dateinamen optional ins Embedding aufnehmen (für name-zentrierte Fragen).
- Tabelleninhalte aus Word/PDF mit erhöhtem Overlap oder dediziertem Tabellen-Chunker indizieren.
- **Skalierung.** SQLite-Setup (Busy-Timeout 30 s, WAL) ist für moderate Parallellast (≤ 10 gleichzeitige Chats) ausgelegt; bei höherer Last Migration auf PostgreSQL.

Diese Dokumentation wurde automatisiert aus der Codebasis `rag-qdrant-local/` (3.036 Python-Zeilen, 17 Module, 9 HTTP-Endpunkte) generiert. Für Detailfragen siehe Inline-Docstrings und `TEST_REPORT.md`.